Clase 7 · kill(), procesos zombie e inicio de cliente-servidor

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 17 sep 2026 · Cierre del Tema 2 e inicio del Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Cómo termina un proceso

Tipo	Cuándo
Muerte natural	Completa su código, llega al final de `main()` y avisa a su padre
Muerte anormal	Una señal externa (Ctrl+C = SIGINT, `kill`) o `abort()` (SIGABRT, 6)

- `kill -9 <PID>` = SIGKILL en x86. Recomendado: **`kill -SIGKILL <PID>`** (nombre de la macro), porque el número cambia entre arquitecturas (en SPARC el 7 o el 10 son otras señales).
- `kill <PID>` sin señal → manda la de terminación.

2. programa10.c: switch/case de señales

- Un `case` por señal (SIGHUP, SIGINT, SIGQUIT, SIGILL, SIGTRAP, SIGABRT, SIGBUS, SIGFPE, SIGTERM, SIGCHLD…), cada uno con `break`; las demás a `default`.
- `signal(SEÑAL, signal_handler)` para cada una: todas responden "Sí puedo manejar…" excepto **SIGKILL (9)**: "No puedo manejar SIGKILL".
- `while(1) { sleep(1); }`; desde otra terminal `kill -SIGHUP <PID>`: el proceso imprime y sigue vivo.
- Si captura SIGINT (2) y SIGTSTP (Ctrl+Z, 20), ya no se puede parar desde esa terminal: hay que usar `kill -9` desde otra.

Error común: un `case SIGKILL` es código inalcanzable.

3. kill() en C: programa11.c

`kill(pid_t pid, int sig)` con `<sys/types.h>` y `<signal.h>`; la macro va sin guion.

- **SIGTERM (15):** se puede capturar o ignorar. **SIGKILL (9):** ineludible.

1. `if ((pid = fork()) == 0)`: el hijo imprime su PID y cada segundo una línea.
2. El padre se salta el `if` y hace `sleep(10)`: el hijo alcanza 10 iteraciones.
3. El padre ejecuta `kill(pid, SIGKILL)` y luego `exit(0)`.

4. Procesos zombie

Definición

Hijo que **ya terminó** pero sigue en la tabla de procesos porque el padre **no ha leído su estado**. Al morir, el kernel manda **SIGCHLD (17 en x86)** al padre; si este duerme u está ocupado, la señal queda encolada.

programa8.c: el hijo hace `exit(0)` de inmediato y el padre `sleep(60)`. Con `ps -ef | grep Carlos` el hijo sale con **Z** (o Z+) y `<defunct>` durante 60 s; al despertar el padre, desaparece.

Estado (`man ps`)	Significado
D	Sueño ininterrumpible (E/S)
I	Inactivo (idle)
R	Ejecutando o listo
S	Sueño interrumpible
T	Detenido (Ctrl+Z)
Z	Zombie (defunct)
X	Muerto: **nunca** debería verse en `ps`

Dos soluciones (tarea moral):

- `programa9_v1.c`: `signal(SIGCHLD, SIG_IGN)` → el sistema limpia al hijo.
- `programa9_v2.c`: manejador de SIGCHLD que llama `wait()` o `waitpid()`.

5. Tema 3: arquitectura cliente-servidor

Netflix, WhatsApp, apps bancarias, redes sociales. Restaurante: **cliente** = comensal, **red** = mesero, **servidor** = cocina.

- **Cliente:** rol activo, inicia con un *request*; recursos limitados; interfaz con el usuario.
- **Servidor:** rol pasivo de escucha; sistemas de alta potencia.
- **Cinco nueves (99.999 %):** nadie da 100 %; AWS, Google Cloud y Oracle garantizan sus SLA sobre 99.999 % (caída 0.001 %).

Socket y puerto (examen)

Socket: cada extremo de un canal de comunicación bidireccional entre dos procesos (como el contacto donde se conecta un foco), de software; misma máquina o en red.

Puerto lógico: campo de **16 bits sin signo** → 2^{16} = **65,536** puertos por host (0–65535). No confundir con puertos físicos (USB, HDMI, VGA).

Pendiente

- Tarea moral: probar programa9_v1 y v2.
- Leer los 8 PDFs del Tema 3 del sitio del profesor (routers, módems, OSI, TCP, puertos, firewall).
- Próxima: crear sockets en C.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Concepto	Para recordar
`kill` en terminal	Usar el nombre (`-SIGKILL`), no el número: portabilidad
SIGKILL / SIGTERM	9, no se captura / 15, se captura o ignora
`kill(pid, sig)`	Padre mata al hijo tras `sleep(10)` (programa11)
Zombie	Hijo muerto, padre no leyó SIGCHLD (17); `ps`: Z, defunct
Evitar zombies	`SIG_IGN` en SIGCHLD o `wait()/waitpid()`
Estado X	Nunca aparece en `ps`

Tema 3

Cliente activo, servidor pasivo; 99.999 %; socket = extremo lógico; puerto = 16 bits → 65,536.